SELECCIÓN DE FRAMEWORK DEL PROYECTO

PROYECTO: DRIVIQUE

INTEGRANTES:
LAURA VANESSA PEREZ PERDOMO
DANNA VALENTINA BARRIOS PENAGOS
EMILY SHARITH AMEZQUITA SAAVEDRA

INSTRUCTOR:
CARLOS JULIO CADENA

TECNOLOGO EN ANALISIS Y DESARROLLO DE SOFTWARE
FICHA: 3145555

SERVICIO NACIONAL DE APRENDIZAJE
CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

2026

Tabla de contenido

1. Introducción ..... **¡Error! Marcador no definido.**
1.1 Propósito del documento 3
1.2 Alcance 3
1.3 Relación con el proyecto formativo 3
2. Análisis de Insumos del Proyecto 3
2.1 Análisis del SRS 3
2.2 Análisis del modelado 5
2.3 Análisis de mockups 5
2.4 Análisis de interacciones 6
3. Selección del Framework Front-End 7
3.1 Alternativas evaluadas 7
3.2 Framework seleccionado 8
3.3 Justificación técnica 9
3.4 Ventajas y limitaciones 10
4. Definición de la Arquitectura Front-End 11
4.1 Tipo de arquitectura seleccionada 11
4.2 Estructura del proyecto (carpetas y módulos) 11
4.3 Flujo de la aplicación 13
4.4 Manejo de estados 13
5. Definición de Patrones de Diseño 15
5.1 Patrones seleccionados 15
5.2 Justificación de uso 15
5.3 Ejemplos de aplicación en el proyecto 15
5.4 Anti-patrones a evitar ..... **¡Error! Marcador no definido.**
6. Integración de la Solución Front-End 16
6.1 Relación con el SRS 16
6.2 Relación con mockups e interacciones ..... **¡Error! Marcador no definido.**
6.3 Relación con el modelado ..... **¡Error! Marcador no definido.**
7. Conclusiones 17
8. Recomendaciones 17
9. Anexos 18
9.1 Diagramas 18
9.2 Estructura de carpetas detallada 19

1. Introducción

1.1 Propósito del documento

Este documento tiene como propósito definir y justificar las decisiones técnicas relacionadas con la selección del framework front-end, la arquitectura del proyecto y los patrones de diseño que se van a implementar en el desarrollo de Drivique Location. Está pensado para que el equipo de desarrollo tenga una guía clara de cómo estructurar el proyecto desde el inicio, tanto para la aplicación web como para la aplicación móvil nativa.

1.2 Alcance

El documento abarca el análisis de los artefactos ya construidos (SRS, mockups del Figma, modelado del sistema e interacciones), y con base en eso define qué framework se va a usar, cómo se va a organizar el proyecto y qué patrones de diseño se van a aplicar. No cubre temas de backend ni bases de datos. El alcance incluye tanto el desarrollo web como el desarrollo móvil nativo para Android, dado que el proyecto requiere una solución unificada para ambas plataformas.

1.3 Relación con el proyecto formativo

Este documento es parte de la formación del Tecnólogo en Análisis y Desarrollo de Software del SENA, ficha 3145555. Se desarrolla como actividad dentro del proyecto formativo Drivique Location, donde el equipo debe tomar decisiones técnicas reales sobre la construcción de una aplicación web y móvil nativa para la gestión de alquiler de vehículos.

2. Análisis de Insumos del Proyecto

Antes de escoger el framework y la arquitectura, se revisaron todos los artefactos que ya se habían construido para el proyecto. Esto es importante porque la tecnología que se elija debe responder a las necesidades reales del sistema, no escogerse al azar.

2.1 Análisis del SRS

El documento SRS de Drivique establece un sistema integral con un alcance específico por plataforma: una aplicación web (web-drivique) que cubre de forma completa el portal para clientes y los paneles administrativos, y una aplicación móvil nativa (app-drivique) optimizada para Android 13 y 14, destinada de forma exclusiva al flujo del cliente final. A continuación, se resumen los puntos más importantes que guiaron la implementación del front-end:

Funcionalidades principales que tiene el sistema:

- Registro e inicio de sesión con autenticación JWT, verificación 2FA (OTP de 6 dígitos) y ruteo basado en roles (cliente, administrador, operador, supervisor).
- Catálogo de vehículos interactivo con búsqueda en tiempo real (debounce) y filtros avanzados multicategoría. • Flujo de reservas (Wizard) multipaso para selección de fechas, horarios, sucursales, servicios adicionales, seguros y confirmación.
- Módulo de pagos integrado con la pasarela Wompi (PSE, tarjetas de crédito/débito y Nequi).
- Sistema de validación de entrega de vehículos mediante un PIN único de 4 dígitos.
- Panel administrativo web robusto para la gestión centralizada de la flota, contratos, pagos, mantenimientos y generación de reportes.
- Generación, descarga y consulta de contratos en PDF desde ambas plataformas.

Requerimientos no funcionales importantes:

- Rendimiento: Tiempos de respuesta óptimos y actualización instantánea de interfaces al interactuar.
- Seguridad: Cumplimiento de OWASP Top 10, uso obligatorio de HTTPS, protección de rutas en el lado del cliente y manejo seguro de tokens JWT.
- Diseño y Accesibilidad: Implementación de Diseño Responsivo (RWD), variables globales para Modo Claro/Oscuro (Light/Dark Mode) y soporte base para internacionalización (i18n).

- Arquitectura y Escalabilidad: Estructura modular basada en componentes, separando estrictamente la presentación visual, el estado global (Zustand/Hooks) y la capa de consumo de servicios (Axios).

2.2 Análisis del modelado

El modelado de datos en el sistema (database_drivique.sql) define las entidades principales con las que interactúa la aplicación. Estas entidades estructuraron directamente los módulos y componentes visuales desarrollados en el front-end (web-drivique y app móvil):

- Usuario (roles de cliente, administrador, encargado y operador)
- Vehículo (catálogo, disponibilidad, características y reseñas)
- Reserva (fechas, sucursales, estado y validación de entrega)
- Contrato (documento legal vinculado a una reserva)
- Pago (asociado a la reserva, pasarela Wompi y efectivo)
- Soporte (tickets de servicio y atención al cliente)
- Reporte (estadísticas generadas por los administradores)

Esto demuestra que el front-end maneja múltiples módulos con lógica propia y datos interrelacionados. Por esta razón, se implementó una arquitectura modular estricta con un manejo de estado centralizado (Zustand y React Query), garantizando que la información se gestione de forma coherente tanto en la interfaz web como en la interfaz móvil nativa.

2.3 Análisis de mockups

Los mockups del proyecto (diseñados en Figma) definieron un ecosistema visual completo de más de 32 pantallas. A la fecha, todas las pantallas han sido diseñadas e implementadas en sus respectivos repositorios (web-drivique y app-drivique), siendo un reflejo exacto del diseño en código. A continuación, se detalla el listado exhaustivo de pantallas estructuradas:

Pantallas diseñadas (web y móvil):

- Pantalla de Landing Page (Pública con propuesta de valor).
- Pantalla de Onboarding / Bienvenida (Exclusiva Móvil).
- Pantalla de Inicio de Sesión (Login).
- Pantalla de Registro de Usuario en múltiples pasos.
- Pantalla de Verificación de Identidad (OTP / 2FA de 6 dígitos).
- Pantalla de Solicitud de Recuperación de Contraseña.
- Pantalla de Restablecimiento de Nueva Contraseña.
- Pantalla Principal del Catálogo de Vehículos (con búsqueda y filtros avanzados).
- Pantalla de Detalle de Vehículo (Galería 360°, especificaciones y reseñas).
- Pantalla de Reserva - Paso 1: Fechas, sucursales y opciones de entrega.
- Pantalla de Reserva - Paso 2: Resumen, desglose de costos y seguros.
- Pantalla de Pago Seguro (Integración con Wompi Checkout y Efectivo).

- Pantalla de Confirmación de Reserva Exitosa.
- Pantalla de "Mis Reservas" (Historial con estados: Activas, Completadas, Canceladas).
- Pantalla de "Mis Favoritos" (Wishlist de vehículos).
- Pantalla de Visualización y Descarga de Contrato (Renderizado PDF).
- Pantalla de Visualización de PIN de Entrega de 4 dígitos.
- Pantalla de Notificaciones y Centro de Alertas.
- Pantalla de Perfil de Usuario y Ajustes de Cuenta.
- Pantalla del Centro de Soporte (FAQ y Formulario de PQR).

Pantallas diseñadas e implementadas (Exclusivas Web Administrativa):

- Pantallas diseñadas e implementadas (Exclusivas Web Administrativa):
- Pantalla del Dashboard Administrativo (Métricas y KPIs principales).
- Pantalla de Gestión de Flota (Listado interactivo de vehículos).
- Pantalla de Creación y Edición de Ficha de Vehículo.
- Pantalla de Gestión de Mantenimientos (Programación y estados).
- Pantalla de Gestión de Reservas y Seguimiento Operativo.
- Pantalla de Gestión de Usuarios, Empleados y Roles.
- Pantalla de Gestión y Auditoría de Contratos.
- Pantalla de Gestión de Pagos, Reembolsos y Transacciones.
- Pantalla de Generación y Descarga de Reportes (PDF, Word y Excel).
- Pantalla de Configuración de Sucursales y Tarifas.

Componentes reutilizables identificados e implementados:

- Tarjetas de vehículo (VehicleCards) reutilizadas en catálogo, detalle y reservas.
- Formularios interactivos con validación en tiempo real.
- Tablas de datos avanzadas (DataTables con paginación y filtros).
- Calendarios de disponibilidad interactivos.
- Modales, alertas de confirmación y botones de acción unificados.
- Barras de navegación (Navbar en web / Bottom tabs y Drawer en móvil).
- Indicadores de estado de carga (Spinners/Skeletons) y error (Toast notifications).

La extensa cantidad de pantallas construidas y la complejidad funcional de los componentes confirman la idoneidad de los frameworks seleccionados (React y React Native). Estos facilitaron la creación de una librería de componentes modulares, permitiendo homologar la lógica y los patrones

2.4 Análisis de interacciones

A partir de los requerimientos y el diseño, se implementaron las siguientes interacciones principales en los respectivos repositorios del front-end:

Navegación y ruteo:

- Flujo del cliente: Autenticación → Inicio (Catálogo) → Detalle de vehículo → Wizard de reserva → Checkout (Pago) → Confirmación.
- Flujos administrativos (Exclusivo Web): Navegación protegida por roles hacia los módulos de gestión (flota, reservas, contratos, reportes).

- Navegación Web (web-drivique): Implementada mediante React Router, garantizando la preservación del historial del navegador (botón "Atrás" funcional) y guardias de ruta (Protected Routes) para evitar accesos no autorizados.
- Navegación Móvil (app-drivique): Implementada mediante Expo Router, utilizando navegación nativa por pestañas (Tabs) y pilas (Stack), con soporte absoluto para los gestos y el botón físico de retroceso en dispositivos Android.

Eventos e interacciones del usuario:

- Búsqueda reactiva (debounce) y filtrado dinámico en el catálogo de vehículos.
- Selección de fechas interactivas en el calendario de disponibilidad del wizard de reservas.
- Llenado y validación de formularios en tiempo real (manejo de errores instantáneo antes del envío).
- Verificación de identidad dinámica mediante código OTP de 6 dígitos.
- Transacciones financieras seguras mediante la integración del checkout de Wompi.
- Solicitud de generación y descarga/visualización interactiva de contratos en PDF.
- Soporte para gestos táctiles nativos en la app móvil (deslizamiento, pulsación larga y scroll fluido).

Flujo general de la aplicación:

El ciclo de vida principal del usuario (cliente) inicia cuando abre la aplicación → se registra y verifica su identidad (OTP) o inicia sesión → navega por el catálogo → selecciona un vehículo → configura su reserva (fechas/sucursales) → realiza el pago → recibe su confirmación y accede a su contrato en PDF.

De manera independiente y paralela, el administrador ejecuta su propio flujo de trabajo exclusivamente desde la interfaz web, ingresando directamente al panel de control (Dashboard) para monitorear y gestionar la operativa completa del sistema. El flujo del cliente es homólogo en ambas plataformas, mientras que la administración aprovecha el formato de escritorio que brinda la web.

3. Selección del Framework Front-End

3.1 Alternativas evaluadas

Se evaluaron tres opciones de frameworks front-end para el desarrollo web y tres para el desarrollo móvil, teniendo en cuenta que el proyecto requiere una solución tanto web como nativa para Android.

Para el desarrollo web:

Angular: Framework completo de Google para aplicaciones web empresariales. Tiene todo integrado (enrutamiento, manejo de formularios, llamadas HTTP, etc.) pero su curva de aprendizaje es alta y puede ser muy complejo para equipos pequeños o en formación.

Vue.js: Framework progresivo y amigable para principiantes. Es más sencillo que Angular y más estructurado que React básico, pero su ecosistema es menos amplio en comparación y tiene menos recursos de aprendizaje disponibles en español.

React (con Vite): Librería de JavaScript de Meta para construir interfaces de usuario basadas en componentes. Es el más usado a nivel mundial, tiene la comunidad más grande y miles de recursos. Se complementa perfectamente con React Native para compartir lógica entre plataformas.

Para el desarrollo móvil nativo:

Flutter: Framework de Google que permite desarrollar apps nativas para Android e iOS con el lenguaje Dart. Aunque produce apps de alto rendimiento, requiere aprender un lenguaje diferente (Dart) y el ecosistema es más pequeño comparado con el mundo JavaScript/React.

Kotlin / Android nativo: Desarrollo completamente nativo para Android. Ofrece el máximo rendimiento pero implica escribir código completamente separado del frontend web, lo que duplica el trabajo para un equipo de dos personas.

React Native (con Expo): Framework de Meta que permite construir aplicaciones móviles nativas usando JavaScript y React, el mismo lenguaje y paradigma que se usa en la web con React. Permite compartir lógica de negocio entre la app web y la app móvil, y genera apps verdaderamente nativas para Android sin necesidad de un navegador embebido.

3.2 Framework seleccionado e implementados

- **Para la aplicación web** (cliente y administración): React empacado con Vite.
- **Para la aplicación móvil nativa** (cliente): React Native estructurado con Expo y Expo Router.

Esta combinación fue seleccionada de forma definitiva porque permitió al equipo unificar el ecosistema bajo un mismo lenguaje base (JavaScript/TypeScript), utilizar el mismo paradigma (basado en componentes) y compartir patrones arquitectónicos. Actualmente, ambos repositorios (web-drivique y app-drivique) operan plenamente bajo esta pila tecnológica.

Herramientas y librerías del stack:

Para la app móvil (ya presentes en el proyecto):

- React Native 0.86.3 + React 19.2.3 → Base del framework móvil nativo actualizado.
- Expo ~57.0 → Plataforma central de desarrollo, empaquetado y acceso a APIs del dispositivo.
- Expo Router ~57.0 → Enrutamiento avanzado basado en sistema de archivos.
- @react-navigation/native + bottom-tabs → Gestión de navegación interna y menú de pestañas inferior.
- react-native-calendars + datetimepicker → Gestión de selección de fechas para reservas en la interfaz nativa.
- react-native-webview → Renderizado de vistas web integradas (utilizado para integraciones como Wompi Checkout).
- react-native-reanimated + gesture-handler → Motor para animaciones nativas fluidas e interacciones táctiles precisas.

Para la web (ya presentes en el proyecto):

- React + Vite → Núcleo de componentes UI y herramienta de compilación ultrarrápida.
- React Router DOM → Sistema de enrutamiento y guardias de protección de rutas para el navegador.
- Tailwind CSS (@tailwindcss/vite) → Framework de utilidades atómicas para diseño responsivo e interfaces modernas.
- React Datepicker + Date-fns → Herramientas para la selección de fechas y manejo del calendario de disponibilidad.
- SweetAlert2 → Manejo de modales dinámicos y alertas de confirmación.
- Recharts → Gráficos interactivos para la visualización de datos en el dashboard administrativo.
- @tanstack/react-query → Motor asíncrono avanzado para caché de peticiones y sincronización de datos con el servidor.

Librerías adicionales comunes a ambas plataformas:

- Zustand → Manejo del estado global de manera directa, ligera y sin boilerplate.

- Axios → Cliente HTTP estándar unificado para la comunicación segura con el backend (API REST).

- i18next + react-i18next → Soporte base de internacionalización preparado para múltiples idiomas en web y móvil.

3.3 Justificación técnica

React Native con Expo se implementó como framework base para la app móvil, y React con Vite para la web, por las siguientes razones operativas y técnicas directamente alineadas con los requisitos de Drivique:

Una sola tecnología para dos plataformas: Tanto React (web) como React Native (móvil) comparten el mismo lenguaje (JavaScript/TypeScript) y el paradigma de desarrollo basado en componentes. Esto garantizó que el equipo no tuviera que dominar tecnologías dispares, permitiendo estructurar una misma arquitectura para el manejo de estado (Zustand) y consumo de datos

App verdaderamente nativa para Android: A diferencia de una PWA (que corre dentro del navegador web), React Native genera una aplicación nativa que se instala directamente en el dispositivo (APK/AAB). Esto garantizó un acceso profundo a las características del teléfono, mejor rendimiento y una experiencia de usuario fluida y familiar.

Expo simplifica el desarrollo: Expo proveyó un conjunto de herramientas y APIs que agilizaron la construcción. Eliminó la necesidad de configurar entornos complejos como Android Studio desde cero, permitió realizar pruebas instantáneas en dispositivos físicos mediante Expo Go y automatizó el empaquetado para distribución.

Expo Router para navegación moderna: La implementación de Expo Router permitió definir las rutas de la app basándose en la estructura de carpetas (un patrón idéntico a Next.js en web). Esto

facilitó el enrutamiento complejo, como los menús inferiores (bottom-tabs) y la protección estricta de rutas de usuario

Componentes reutilizables: El diseño contempló patrones visuales recurrentes (tarjetas de vehículo, formularios de pago, tablas). React permitió que estos componentes se desarrollaran de manera aislada e independiente, facilitando adaptar sus lógicas entre la versión web y móvil sin reinventar la rueda.

Compatibilidad con Android 13 y 14: React Native y Expo SDK 57 (la versión que actualmente corre en el proyecto) están completamente optimizados para soportar de manera nativa las versiones recientes del sistema operativo exigidas por el SRS

Comunidad y recursos: React Native es el framework más usado para desarrollo móvil con JavaScript. Cuenta con miles de tutoriales, librerías adicionales y soporte activo de la comunidad, lo que facilita encontrar soluciones durante el desarrollo.

Integración con APIs REST: El backend de Drivique se diseñó como una API REST. La combinación del ecosistema React con Axios facilitó el consumo seguro de estos servicios y el manejo de respuestas asíncronas de manera casi idéntica en ambas plataformas.

3.4 Ventajas y limitaciones

Ventajas de React + Vite (Web) y React Native + Expo (Móvil) para este proyecto:

- App móvil verdaderamente nativa para Android 13 y 14 (no depende del navegador).
- Mismo lenguaje base (JavaScript/TypeScript) para web y móvil, lo que reduce la curva de aprendizaje del equipo.
- El tamaño del instalable (APK) generado con Expo puede ser ligeramente mayor al de una app nativa pura, aunque no afecta la funcionalidad.
- Lógica de negocio ampliamente compartible entre web y móvil (servicios con Axios, validaciones, estado con Zustand).
- Vite proporciona tiempos de compilación y carga casi instantáneos en la versión web.
- El uso de Tailwind CSS en la web permite un diseño ágil y responsivo.
- Expo simplifica enormemente la configuración nativa, pruebas físicas y distribución de la app móvil.
- Expo Router proporciona un sistema de navegación moderno, basado en archivos, fácil de organizar.
- Gran ecosistema de librerías compatibles tanto para React como para React Native.
- react-native-web permite probar componentes de la app móvil en el navegador durante el desarrollo.
- Compatibilidad total con las pasarelas de pago (ej. SDK y checkout de Wompi).
- Soporte a gestos táctiles nativos con react-native-gesture-handler.
- Animaciones de alto rendimiento con react-native-reanimated.

Limitaciones a tener en cuenta:

- Algunas librerías de la web no tienen equivalente directo en móvil (por ejemplo, la generación y visualización de PDF obligó a usar librerías nativas específicas como expo-print).

- Los estilos en React Native usan una sintaxis similar pero diferente a CSS (no se puede usar Tailwind directamente en la app móvil, obligando a adaptar los diseños a StyleSheet).
- El tamaño del instalable (APK) generado con Expo puede ser ligeramente mayor al de una app nativa pura, aunque no afecta la funcionalidad.
- En la versión web, la seguridad de la sesión depende del navegador, mientras que en móvil requiere configurar esquemas de URL y almacenamiento local protegido (usando SecureStore de Expo en lugar de localStorage).

4. Definición de la Arquitectura Front-End

4.1 Tipo de arquitectura seleccionada

Se implementó una arquitectura basada en componentes por módulos, aplicada de forma coherente tanto a la aplicación web (React + Vite) como a la aplicación móvil nativa (React Native + Expo). Esto significa que la aplicación se estructuró en módulos según las funcionalidades del sistema (autenticación, catálogo, reservas, pagos, panel admin, etc.), y dentro de cada módulo se crean los componentes, rutas y lógicas necesarias.

Esta arquitectura es ideal porque:

- Permite que cada módulo sea desarrollado y probado de forma independiente.
- Facilita el trabajo en equipo, ya que cada integrante puede trabajar en un módulo sin afectar el código del otro.
- Es altamente escalable: si en el futuro se necesita agregar un nuevo módulo, se puede hacer sin alterar lo que ya existe.
- Permite abstraer y compartir la mayor cantidad posible de lógica (servicios y estados) entre la app web y la app móvil, reduciendo drásticamente la duplicación de código.4.2 Estructura del proyecto (carpetas y módulos)

Estructura de la app móvil (React Native + Expo — basada en el proyecto existente):

La estructura del proyecto app-drivique sigue un estricto patrón modular y utiliza el enrutamiento basado en archivos de Expo Router. A diferencia de la plataforma web, la aplicación móvil carece deliberadamente de módulos administrativos, enfocándose de manera exclusiva en el flujo del cliente final. A continuación se detalla el árbol arquitectónico definitivo implementado:

```
app-drivique/
├── app/                          → Rutas de la aplicación (Expo Router)
|   ├── (tabs)/                   → Navegación principal inferior (Catálogo, Reservas, Perfil)
|   ├── (auth)/                   → Rutas de autenticación (Login, Registro, OTP)
|   ├── onboarding/               → Flujo de bienvenida interactivo para nuevos usuarios
|   ├── vehicle/                  → Flujo de visualización de detalles del vehículo y reserva
|   ├── contract-view.tsx         → Pantalla para renderizado y descarga de contrato en PDF
|   ├── wompi-checkout.tsx        → Integración nativa mediante WebView para pasarela Wompi
|   ├── payment-response.tsx      → Procesamiento del estado de las transacciones
|   ├── modal.tsx                 → Estructura global para ventanas modales
|   ├── index.tsx                 → Punto de entrada y redireccionamiento inicial
|   └── _layout.tsx               → Proveedor de contexto y Layout raíz de la app
├── components/                   → Librería de componentes visuales (Botones, Tarjetas, Inputs)
├── modules/                      → Encapsulamiento de lógica de negocio y servicios
|   ├── auth/                     → Lógica de autenticación y sesión
|   ├── catalog/                  → Lógica de búsqueda y filtrado de flota
|   ├── i18n/                     → Configuración y diccionarios de internacionalización
|   ├── notifications/            → Lógica de alertas push y centro de mensajes
|   ├── onboarding/               → Lógica de control de primera apertura
|   ├── profile/                  → Lógica de gestión de cuenta del usuario
|   ├── reservation/              → Lógica del wizard de reservas y validaciones
|   └── support/                  → Lógica de tickets y PQR
├── store/                        → Configuración del estado global centralizado (Zustand)
├── services/                     → Configuración del cliente HTTP y llamadas API base (Axios)
├── hooks/                        → Custom hooks compartidos (ej. useColorScheme)
├── utils/                        → Funciones de utilidad pura y formateo de datos
├── constants/                    → Variables globales, configuración de tema (Colors, Fonts)
├── mocks/                        → Datos simulados para pruebas ágiles de componentes
├── assets/                       → Recursos estáticos (Imágenes, Fuentes, Íconos)
├── app.json                      → Configuración y manifiesto de construcción de Expo
└── package.json                  → Declaración de dependencias del proyecto
```

Cada carpeta funcional dentro de modules/ se diseñó para contener internamente su propia cohesión, siguiendo este patrón:

```
nombre_del_modulo/
├── components/    → Componentes visuales estrictamente específicos de este módulo
├── hooks/         → Lógica y reglas de negocio del módulo
├── services/      → Endpoints y llamadas a la API correspondientes a esta entidad
└── types/         → Interfaces y tipados estrictos (TypeScript)
```

Estructura de la app web (React + Vite):

```
web-drivique/
├── src/                        → Código fuente de la aplicación
|   ├── assets/                 → Recursos estáticos (imágenes, SVG, logos)
|   ├── components/             → Componentes visuales genéricos y reutilizables
|   ├── contexts/               → Proveedores de contexto de React (Context API)
|   ├── data/                   → Datos estáticos y variables fijas
|   ├── hooks/                  → Custom hooks globales compartidos
|   ├── i18n/                   → Configuración y diccionarios de internacionalización
|   ├── mocks/                  → Información simulada para pruebas locales
|   ├── modules/                → Núcleo de la app (separación por dominios de negocio)
|   |   ├── admin/              → Panel administrativo completo
|   |   ├── auth/               → Autenticación, registro y validación (OTP)
|   |   ├── catalog/            → Catálogo público de vehículos y filtros
|   |   ├── contracts/          → Generación, firma y visor de contratos
|   |   ├── landing/            → Página principal pública
|   |   ├── notifications/      → Alertas y bandeja de mensajes
|   |   ├── payments/           → Flujos de pago y checkout
|   |   ├── profile/            → Gestión de cuenta del cliente
|   |   ├── reservations/       → Wizard de reservas y disponibilidad
|   |   └── support/            → Centro de ayuda y PQR
|   ├── routes/                 → Enrutamiento centralizado (React Router DOM)
|   ├── services/               → Configuración global de Axios y endpoints
|   ├── store/                  → Estado global centralizado (Zustand)
|   ├── styles/                 → Estilos globales y utilidades de Tailwind CSS
|   ├── utils/                  → Funciones de utilidad pura y formateo
|   ├── App.jsx                 → Componente raíz de la interfaz
|   └── main.jsx                → Punto de entrada y montaje de React
├── .env                        → Variables de entorno locales
├── index.html                  → Plantilla HTML principal
├── package.json                → Dependencias del proyecto web
├── tailwind.config.js          → Configuración de diseño de Tailwind CSS
└── vite.config.js              → Configuración del empaquetador ultrarrápido
```

Cada carpeta funcional dentro de modules/ se diseñó para contener internamente su propia cohesión, siguiendo este patrón:

```
nombre_del_modulo/
├── components/    → Componentes visuales específicos de la vista
├── pages/         → Vistas completas que se inyectan en el enrutador
├── hooks/         → Lógica de estado local del módulo
└── services/      → Peticiones API exclusivas del dominio
```

4.3 Flujo de la aplicación

El ciclo de vida y la gestión de sesión de la aplicación funcionan de forma equivalente en web y móvil, pero el enrutamiento respeta estrictamente las restricciones de rol por plataforma (el panel administrativo es exclusivo de la web):

App móvil (React Native + Expo Router - Flujo Cliente):

- El usuario abre la app en su dispositivo móvil (y visualiza el onboarding si es su primera vez).

- Expo Router verifica en segundo plano si existe un token JWT almacenado de forma cifrada en Expo SecureStore.
- Si no hay sesión iniciada, el sistema enruta automáticamente al stack de autenticación (Login, Registro, validación OTP).
- Tras la autenticación exitosa, el token se guarda de forma segura y el perfil de usuario se inyecta en el estado global (Zustand).
- La aplicación habilita la navegación inferior (Tabs) exclusiva para el cliente.
- El cliente navega de forma fluida por el catálogo interactivo, configura reservas, realiza pagos integrados y visualiza contratos en PDF.
- Nota de arquitectura: La app móvil bloquea por diseño las sesiones y accesos con rol administrativo.

App web (React + Vite + React Router - Flujo Híbrido):

- El usuario ingresa a la plataforma web a través del navegador.
- React Router evalúa el estado global y la persistencia del JWT en el almacenamiento local.
- Si no tiene sesión, puede navegar por el catálogo público (landing), pero es redirigido a la pantalla de login para acciones transaccionales.
- Tras iniciar sesión, las guardias de ruta (Protected Routes) evalúan el rol del usuario:

4.4 Manejo de estados

La gestión de la información (estado) se estructuró en diferentes niveles estratégicos y escalables, aplicables según la necesidad de la versión web y móvil:

Estado local de Interfaz (useState / useReducer): Se utiliza estrictamente para datos efímeros de UI que solo interesan dentro del alcance de un único componente (ej. el texto de un input de búsqueda, abrir/cerrar un menú desplegable, o mostrar un spinner de carga local).

Estado del Servidor y Caché Asíncrona (@tanstack/react-query): Para la versión web, se implementó React Query como gestor del estado del servidor. Esto permite mantener sincronizados los datos que provienen del backend (ej. catálogo de vehículos en tiempo real), automatizando la gestión de caché, las revalidaciones y los reintentos, evitando sobrecargar el estado global manual.

Estado global de Aplicación (Zustand / Context API): Se implementó Zustand como almacén central ultraligero para datos críticos que deben persistir y compartirse a través de todos los módulos y pantallas (ej. los datos del usuario autenticado, el vehículo temporal seleccionado en el wizard de reservas, y configuraciones de idioma i18n). Al ser compatible con React y React Native, los stores lógicos de Zustand se pueden homologar entre ambas plataformas.

Almacenamiento Persistente por plataforma: Para mantener la sesión viva y guardar preferencias incluso si el usuario cierra completamente la aplicación, se implementaron las soluciones nativas correspondientes:

- En Móvil (app-drivique): Se implementó Expo SecureStore para el cifrado y guardado seguro del token JWT y variables sensibles, complementado con @react-native-async-storage para persistencia de datos menores o configuraciones locales.

- En Web (web-drivique): Se hace uso de la API nativa del navegador (localStorage / sessionStorage) para inyectar y persistir el token de sesión y recuperar el estado de Zustand al recargar la página.

5. Definición de Patrones de Diseño

5.1 Patrones seleccionados

Se implementaron los siguientes patrones de diseño de ingeniería de software en el proyecto. A continuación se muestra el nombre formal del patrón y, entre paréntesis, su implementación práctica en el ecosistema de React (Web) y React Native (Móvil):

- Composite → Componentes inteligentes y presentacionales (Smart/Dumb Components)
- Template Method → Ganchos personalizados (Custom Hooks)
- Facade → Capa de Servicios centralizada (Service Layer)
- Strategy → Control de Acceso y Rutas Protegidas (Protected Routes / Layouts)
- Observer → Suscripción y manejo de estado global (Zustand)

5.2 Justificación de uso

Composite → Componentes Inteligentes y Tontos: El sistema requería múltiples componentes visuales modulares (tarjetas de vehículo, tablas de datos, modales, formularios de pago) que se repiten en varios flujos. Al separar la lógica de la presentación, el mismo componente visual puede reutilizarse inyectándole datos diferentes sin duplicar código. Un componente "tonto" (Presentational) solo recibe datos por props y renderiza la UI; un componente "inteligente" (Container) obtiene los datos (del store de Zustand o de la API) y se los provee al componente tonto.

Template Method → Custom Hooks: Existen lógicas transversales que se repiten en múltiples módulos, como la validación de formularios, el manejo de temas oscuro/claro, llamadas a la API y manejo del estado de carga/error (Loaders). Los custom hooks implementados en las carpetas hooks/ permitieron extraer esta lógica a un archivo aislado e invocarla donde sea necesario. La lógica de estos hooks es casi idéntica entre React y React Native, facilitando la homologación.

Facade → Capa de Servicios: Todas las llamadas asíncronas al backend (API REST) se ejecutan exclusivamente desde los archivos dentro de las carpetas services/ mediante interceptores de Axios, y jamás directamente desde la UI de los componentes. Esto crea una "fachada" que centraliza la comunicación con el servidor, haciendo que sea trivial cambiar la URL base, refrescar tokens JWT o manejar errores globales (como un Error 401) en un solo archivo para toda la aplicación.

Strategy → Control de Acceso y Rutas Protegidas: El SRS exige autenticación basada en JWT con diferentes niveles de autorización. En la app web (React Router), se inyectan "Guardias de Ruta" condicionales (Estrategias) que verifican si el usuario tiene sesión activa y si su rol corresponde (ej. denegando el paso de un cliente al panel administrativo). En la app móvil (Expo Router), la misma estrategia de evaluación se aplica en los Layouts, expulsando automáticamente a la pantalla de Login a cualquier usuario que pierda su token de sesión.

Observer → Manejo de estado global (Zustand): La librería Zustand implementa de forma reactiva el patrón Observer. Cuando el estado global muta (por ejemplo, el usuario cambia el idioma

i18n, o inicia sesión), el store notifica el cambio y todos los componentes de la interfaz que están "suscritos" u "observando" dicha porción del estado se re-renderizan automáticamente para reflejar el cambio. Zustand probó ser 100% compatible y funcional tanto en el DOM de la web como en la vista nativa del móvil.5.3 Ejemplos de aplicación en el proyecto

6. Integración de la Solución Front-End

6.1 Relación con el SRS

- Cada Cada módulo funcional implementado en el front-end responde directamente y con exactitud a los requerimientos del SRS:
- RF de autenticación: Módulo auth/ con rutas protegidas por JWT. En móvil, el token persiste estrictamente en Expo SecureStore; en web, se almacena en el navegador y se inyecta en el cliente Axios.
- RF de catálogo: Módulo catalog/ con tarjetas de vehículo unificadas y lógicas de búsqueda reactiva (debounce), adaptadas a los gestos de cada plataforma.
- RF de reservas: Módulo reservation/ con un wizard multipaso y gestión interactiva de fechas y disponibilidad de sucursales.
- RF de pagos (Wompi): Integración nativa implementada. En la app móvil, se ejecuta el checkout mediante react-native-webview para cumplir con las políticas de seguridad; en web, se embebe el SDK/Widget directo de Wompi.
- RF de contratos: Módulo de visualización en PDF. En la app móvil se solucionó de manera nativa mediante expo-print y utilidades de renderizado; en la web, de forma directa desde el navegador.
- RF de panel administrativo: Módulo admin/ con dashboards, gestión de flota, usuarios y mantenimientos. Este módulo es de acceso exclusivo en la plataforma web, bloqueado por diseño en la versión móvil.
- RF de reportes: Sección exclusiva dentro del panel web para la generación dinámica de exportables (PDF, Word, Excel). Los requerimientos no funcionales (RNF) también se cubrieron con éxito:
- Seguridad: El JWT se transporta como Bearer Token en las cabeceras HTTP globales mediante interceptores de Axios.
- Rendimiento: Vite y Expo optimizan y minifican el código para producción. Expo Router soporta carga diferida (lazy loading) de pantallas en móvil para ahorrar memoria.
- Accesibilidad: React Native hace uso de los atributos nativos de Android; en la web se aprovecharon las clases y contrastes definidos en Tailwind CSS.
- Compatibilidad: El uso de Expo SDK 57 (versión actual del proyecto) garantiza soporte nativo total para Android 13 y 14, tal como lo exige el cliente.

6.2 Relación con mockups e interacciones:

- Cada pantalla diseñada en Figma corresponde hoy a una ruta activa dentro de Expo Router (en móvil) o a una vista inyectada en React Router DOM (en web).
- Los componentes visuales que se repetían en Figma (tarjetas, inputs, tablas) se consolidaron en la carpeta global components/ y se reutilizaron en toda la app.
- La transición entre vistas obedece de forma fluida a las interacciones pensadas en el diseño, aplicando gestos táctiles reales en la versión de Android y clics convencionales en escritorio.
- Todos los estados visuales diseñados (Cargando Skeleton, Error, Vacío) se controlaron dinámicamente conectando los Custom Hooks con el estado asíncrono de las peticiones.

6.3 Relación con el modelado (Base de Datos)

- Cada esquema de la base de datos (iam, fleet, rental, billing) tiene su contraparte como un archivo de servicio en services/ (ej. authService.ts, vehicleService.ts, paymentService.ts), reutilizable por ambas plataformas.
- Los objetos y formularios (payloads) que el front-end envía coinciden con la validación estricta de las tablas, previniendo errores de tipeo o de integridad referencial.
- Las relaciones complejas entre entidades (ej. un "Contrato" que depende de una "Reserva", que a su vez depende de un "Vehículo" y un "Usuario") se abstrajeron agrupando la información centralizada en el estado global de Zustand antes de pintarlas en la pantalla.

7. Conclusiones

Después de analizar y desarrollar todos los artefactos del proyecto, se llegó a las siguientes conclusiones:

- React Native con Expo probó ser la tecnología más adecuada para la app móvil de Drivique, logrando una aplicación verdaderamente nativa para Android 13 y 14 con un rendimiento superior. El proyecto opera de manera estable sobre esta tecnología (Expo SDK 57, React Native 0.86.3, TypeScript).
- React con Vite resultó ser la solución ideal para la plataforma web. Ambas soluciones (React web + React Native móvil) comparten el mismo ecosistema (JavaScript/TypeScript) y paradigma de componentes, lo que permitió que un equipo pequeño gestionara ambas plataformas de forma ágil y unificada.
- La arquitectura modular por funcionalidades facilitó enormemente el desarrollo, haciendo el código escalable y fácil de mantener tanto en web como en móvil.
- Los patrones de diseño implementados (Smart/Dumb components, Custom hooks, Protected Routes y Service Layer) respondieron con éxito a las necesidades del proyecto, permitiendo reutilizar código para las más de 32 pantallas construidas y blindar la seguridad con JWT.

8. Recomendaciones

- Mantener estrictamente actualizada la carpeta de componentes compartidos (components/) antes de escalar nuevos flujos, garantizando la consistencia visual.
- Figma como fuente de verdad: Cualquier nueva pantalla o funcionalidad solicitada a futuro debe ser prototipada primero en Figma antes de codificarse, manteniendo el orden de trabajo ya establecido.
- Capa de Servicios aislada: Mantener la carpeta services/ completamente independiente de la UI. Cualquier nuevo endpoint creado en el backend debe registrarse aquí.
- Tipado Estricto: Exigir siempre la creación de interfaces (TypeScript) al definir nuevas respuestas de la API para prevenir errores en tiempo de ejecución.
- Pruebas Nativas: Realizar pruebas periódicas en dispositivos físicos Android reales usando Expo Go tras añadir nuevas librerías, y compilar el APK de forma regular.

- Seguridad Móvil: Monitorear las actualizaciones de seguridad de Expo SecureStore para garantizar que la sesión JWT siga cifrada bajo los estándares más altos en futuras versiones de Android.

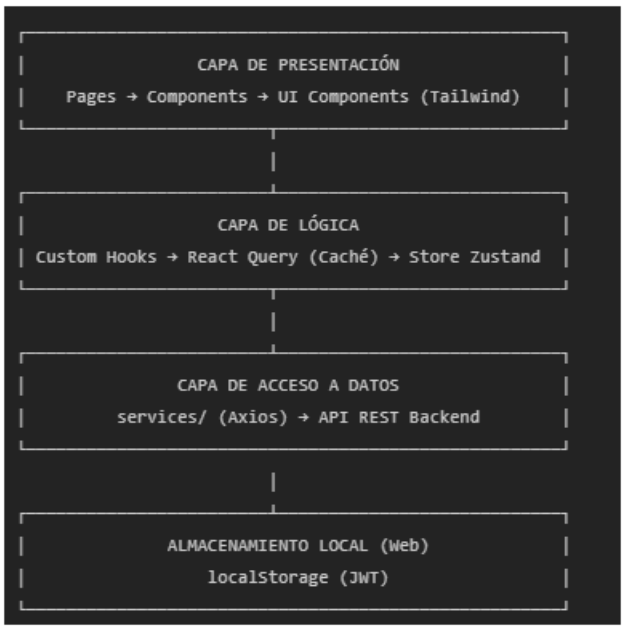
9. Anexos

9.1 Diagramas

Diagrama de arquitectura — App Móvil (React Native + Expo):



Diagrama de arquitectura — App Web (React + Vite):



9.2 Estructura de carpetas detallada

Stack tecnológico completo — App Móvil:

- Tecnologías ya presentes e implementadas en el proyecto (package.json):
- react 19.2.3 + react-native 0.86.3 → Base del framework nativo.
- expo ~57.0 → Plataforma central de desarrollo y acceso a APIs nativas.
- expo-router ~57.0 → Enrutamiento basado en archivos (file-based routing).
- @react-navigation/native + @react-navigation/bottom-tabs → Navegación nativa por pestañas.
- react-native-reanimated + react-native-gesture-handler → Animaciones fluidas y gestos táctiles.
- react-native-webview → Renderizado seguro de vistas integradas (ej. pasarela Wompi).
- zustand → Estado global centralizado. • axios → Peticiones HTTP hacia la API REST.
- expo-secure-store → Almacenamiento cifrado y seguro del token JWT.
- typescript ~5.9.2 → Tipado estático estricto para prevención de errores. Stack tecnológico completo

Stack tecnológico completo — App Móvil:

- Tecnologías ya presentes e implementadas en el proyecto (package.json):
- react 19.2.5 + vite 8.0.10 → Núcleo de componentes web y compilador ultrarrápido.
- react-router-dom → Sistema de enrutamiento y guardias de ruta para el navegador.
- tailwindcss (@tailwindcss/vite) → Framework de utilidades atómicas para diseño ágil y responsivo.
- @tanstack/react-query → Motor asíncrono para caché de peticiones y sincronización de datos.
- zustand → Manejo de estado global local. • axios → Peticiones HTTP hacia la API REST.
- react-datepicker + date-fns → Interacción y manejo preciso del calendario de reservas.
- recharts → Generación de gráficos dinámicos para el panel administrativo.
- sweetalert2 → Renderizado de modales y alertas dinámicas unificadas.

Clasificación de patrones de diseño usados:

Patrones Estructurales (cómo se organizan los elementos):

- Composite: Los componentes se anidan dentro de otros componentes formando un árbol visual. Por ejemplo, la pantalla del Wizard de reservas contiene un componente Datepicker (Calendario), que a su vez contiene celdas de días que el usuario puede seleccionar. Esto aplica igual en web y en móvil.
- Facade (Fachada): La capa de servicios (carpeta services/) actúa como fachada. Oculta toda la complejidad de la configuración de Axios, las URLs de la API, los interceptores y el manejo de tokens JWT detrás de funciones limpias y simples como getVehiculos() o crearReserva().

Patrones de Comportamiento (cómo interactúan los elementos):

- Observer: Zustand implementa este patrón. Cuando una variable del estado global cambia (ej. al iniciar sesión), el store notifica a todos los componentes que están "observando" ese estado y estos se re-renderizan automáticamente en web y en móvil.
- Strategy: Las guardias de ruta y Layouts implementan este patrón. La "estrategia" de navegación cambia dinámicamente según dos factores: en móvil (Expo Router), la estrategia bloquea o permite el paso dependiendo únicamente de si el usuario tiene una sesión activa; en web (React Router), la estrategia evalúa el rol del usuario (cliente o administrador) para mostrar u ocultar los módulos correspondientes.
- Template Method: Los custom hooks (useReservas, useVehiculos, useAuth) siguen una estructura asíncrona base similar: solicitar datos → manejar el estado de carga (Loading) → capturar posibles fallos (Error) → retornar el resultado. Cada hook personaliza los pasos internamente según su necesidad. La implementación de esta lógica es idéntica en web y en móvil.